

CYBERSECURITY INTERNSHIP

Organization name: Elevate labs

Student Name: Malladi Siva Shankar

University: Aditya University

Domain: Cybersecurity

Duration: 1st January 2026 to 30th April 2026

Task 7: Web Application Vulnerability Testing

1. OWASP Top 10 Vulnerabilities :-

The OWASP Top 10 is a globally recognized document published by the Open Web Application Security Project (OWASP). It identifies the most critical security risks to web applications based on real-world data, impact, and exploitability. It helps developers, testers, and security professionals understand and prevent common web attacks.

1. Broken Access Control

Broken Access Control occurs when applications fail to properly restrict what authenticated users are allowed to do. Attackers can access unauthorized data or perform restricted actions such as viewing other users' data or gaining administrative privileges.

2. Cryptographic Failures

This vulnerability arises when sensitive data like passwords, credit card numbers, or personal information is not properly encrypted or protected. Weak encryption algorithms, poor key management, or transmitting data in plain text can lead to data exposure.

3. Injection

Injection flaws occur when untrusted user input is sent to an interpreter without proper validation. Common examples include SQL injection and command injection. Attackers can manipulate queries to access, modify, or delete data.

4. Insecure Design

Insecure Design refers to fundamental design flaws in an application's architecture. These issues cannot be fixed by implementation alone. Examples include lack of rate limiting, weak business logic, or absence of threat modeling during development.

5. Security Misconfiguration

Security Misconfiguration happens when systems are configured insecurely. This includes using default credentials, unnecessary open ports, verbose error messages, or improperly configured cloud services, making systems easy targets.

6. Vulnerable and Outdated Components

Applications often rely on third-party libraries and frameworks. If these components contain known vulnerabilities and are not updated, attackers can exploit them to compromise the application.

7. Identification and Authentication Failures

This vulnerability involves weak authentication mechanisms such as poor password policies, improper session management, or missing multi-factor authentication. It allows attackers to impersonate legitimate users.

8. Software and Data Integrity Failures

This occurs when applications do not verify the integrity of software updates, libraries, or data. Attackers can introduce malicious code through compromised updates or CI/CD pipelines, leading to supply-chain attacks.

9. Security Logging and Monitoring Failures

When applications fail to log security events or monitor suspicious activities, attacks may go undetected. This delays incident response and increases damage during breaches.

10. Server-Side Request Forgery (SSRF)

SSRF occurs when a server is tricked into making unauthorized requests to internal or external systems. Attackers can access internal services, cloud metadata, or sensitive resources not intended to be exposed.

2. Setup vulnerable app (DVWA / Juice Shop)

Vulnerable applications are intentionally insecure platforms used for learning, testing, and understanding web application vulnerabilities, especially the OWASP Top 10. Two widely used platforms are DVWA (Damn Vulnerable Web Application) and OWASP Juice Shop.

1. DVWA (Damn Vulnerable Web Application)

DVWA is a PHP/MySQL-based web application designed to help students and security professionals understand common web vulnerabilities in a controlled lab environment.

DVWA runs on a local web server stack. Once the server environment is installed:

- DVWA files are placed inside the web server directory
- Database configuration is set in a config file
- The application initializes its own database
- Security levels (Low, Medium, High) can be configured

Purpose of DVWA

- Demonstrates vulnerabilities such as SQL Injection, XSS, CSRF, File Inclusion
- Helps understand **how vulnerabilities exist**, not how to exploit real systems

STEP 1: Install required packages Give

command as `sudo apt update`

`sudo apt install apache2 mariadb-server php php-mysqli php-gd git -y`

```
(kali@kali)-[~]
$ sudo apt update
sudo apt install apache2 mariadb-server php php-mysqli php-gd git -y
[sudo] password for kali:
Hit:1 http://http.kali.org/kali kali-rolling InRelease
1974 packages can be upgraded. Run 'apt list --upgradable' to see them.
Note, selecting 'php8.4-mysql' instead of 'php-mysqli'
apache2 is already the newest version (2.4.66-2+b1).
apache2 set to manually installed.
mariadb-server is already the newest version (1:11.8.5-4).
mariadb-server set to manually installed.
php is already the newest version (2:8.4+99).
php set to manually installed.
php8.4-mysql is already the newest version (8.4.16-1).
php8.4-mysql set to manually installed.
php-gd is already the newest version (2:8.4+99).
git is already the newest version (1:2.51.0-1).
git set to manually installed.
The following package was automatically installed and is no longer required:
  libconfig-inifiles-perl
Use 'sudo apt autoremove' to remove it.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 1974
```

STEP 2: Start & enable services

`sudo systemctl start apache2 mariadb sudo`

`systemctl enable apache2 mariadb`

```
(kali@kali)-[~]
$ sudo systemctl start apache2 mariadb
sudo systemctl enable apache2 mariadb
Synchronizing state of apache2.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable apache2
Synchronizing state of mariadb.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable mariadb

(kali@kali)-[~]
```

Check:

`systemctl status apache2 systemctl`

`status mariadb`

```
(kali@kali)-[~]
$ systemctl status apache2
systemctl status mariadb
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: disabled)
   Active: active (running) since Tue 2026-01-27 07:06:25 EST; 5min ago
 Invocation: 42187261417546b4922347893dd34797
    Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 693 (apache2)
  Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served/sec: 0 B/sec"
    Tasks: 6 (limit: 2208)
  Memory: 24.7M (peak: 25M)
     CPU: 238ms
   CGroup: /system.slice/apache2.service
           └─693 /usr/sbin/apache2 -k start -DFOREGROUND
             └─825 /usr/sbin/apache2 -k start -DFOREGROUND
               └─826 /usr/sbin/apache2 -k start -DFOREGROUND
                 └─827 /usr/sbin/apache2 -k start -DFOREGROUND
                   └─828 /usr/sbin/apache2 -k start -DFOREGROUND
                     └─829 /usr/sbin/apache2 -k start -DFOREGROUND

Jan 27 07:06:24 kali systemd[1]: Starting apache2.service - The Apache HTTP Server ...
Jan 27 07:06:24 kali apache2[693]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set the 'ServerName' directive globally to su>
Jan 27 07:06:25 kali systemd[1]: Started apache2.service - The Apache HTTP Server.
lines 1-21/21 (END)
```

STEP 3: Download DVWA

```
cd /var/www/html sudo git clone
```

```
https://github.com/digininja/DVWA.git
```

```
(kali㉿kali)-[~]  
$ cd /var/www/html  
$ sudo git clone https://github.com/digininja/DVWA.git  
fatal: destination path 'DVWA' already exists and is not an empty directory.
```

STEP 4: Create database & user (IMPORTANT)

```
sudo mysql
```

Inside MariaDB:

```
CREATE DATABASE dvwa;
```

```
CREATE USER 'dvwa'@'localhost' IDENTIFIED BY 'dvwa';
```

```
GRANT ALL PRIVILEGES ON dvwa.* TO 'dvwa'@'localhost';
```

```
FLUSH PRIVILEGES;
```

```
EXIT;
```

```
(kali㉿kali)-[/var/www/html]  
$ sudo mysql  
Welcome to the MariaDB monitor.  Commands end with ; or \g.  
Your MariaDB connection id is 31  
Server version: 11.8.5-MariaDB-4 from Debian -- Please help get to 10k stars at https://github.com/MariaDB/Server  
  
Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.  
  
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.  
  
MariaDB [(none)]> CREATE DATABASE dvwa;  
ERROR 1007 (HY000): Can't create database 'dvwa'; database exists  
MariaDB [(none)]> CREATE USER 'dvwa'@'localhost' IDENTIFIED BY 'dvwa';  
ERROR 1396 (HY000): Operation CREATE USER failed for 'dvwa'@'localhost'  
MariaDB [(none)]> GRANT ALL PRIVILEGES ON dvwa.* TO 'dvwa'@'localhost';  
Query OK, 0 rows affected (0.003 sec)  
  
MariaDB [(none)]> FLUSH PRIVILEGES;  
Query OK, 0 rows affected (0.001 sec)  
  
MariaDB [(none)]> EXIT;  
Bye
```

STEP 5: Configure DVWA

```
cd /var/www/html/DVWA/config sudo cp
```

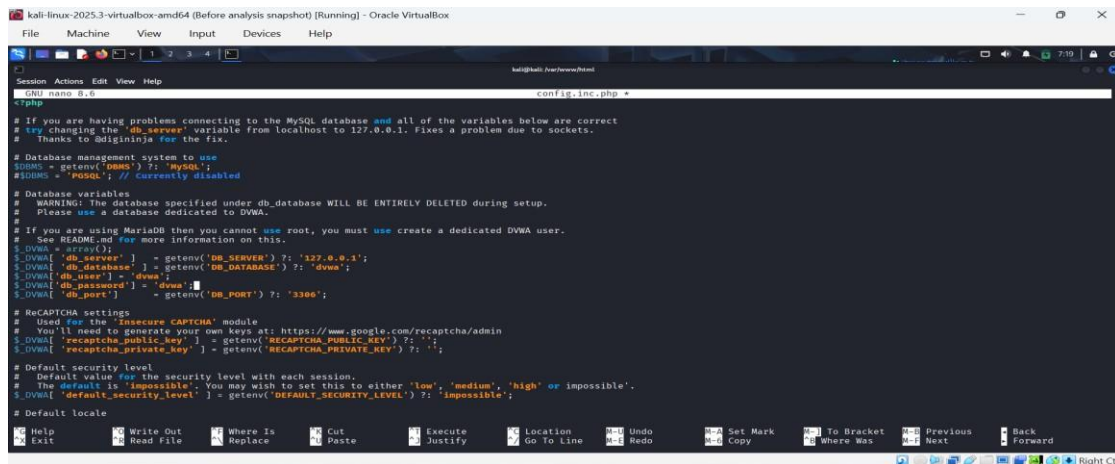
```
config.inc.php.dist config.inc.php sudo
```

```
nano config.inc.php
```

Edit ONLY these lines:

```
$_DVWA['db_user'] = 'dvwa';
```

```
$_DVWA['db_password'] = 'dvwa';
```



```
kali@kali: /var/www/html$ nano config.inc.php
# If you are having problems connecting to the MySQL database and all of the variables below are correct
# try changing the 'db_server' variable from localhost to 127.0.0.1. Fixes a problem due to sockets.
# Thanks to @digininja for the fix.

# Database management system to use
$dbms = getenv('dbms') ? 'mysql' : 'mysql';
# $dbms = 'pgsql'; // Currently disabled

# Database variables
# WARNING: The database specified under db_database WILL BE ENTIRELY DELETED during setup.
# Please use a database dedicated to DVWA.
# If you are using MariaDB then you cannot use root, you must use create a dedicated DVWA user.
# See README.md for more information on this.
$_DVWA['db_server'] = getenv('DB_SERVER') ? '127.0.0.1' : 'localhost';
$_DVWA['db_database'] = getenv('DB_DATABASE') ? 'dvwa' : 'dvwa';
$_DVWA['db_user'] = 'dvwa';
$_DVWA['db_password'] = 'dvwa';
$_DVWA['db_port'] = getenv('DB_PORT') ? '3306' : '3306';

# ReCAPTCHA settings
# Used for the 'Insecure CAPTCHA' module
# You'll need to generate your own keys at: https://www.google.com/recaptcha/admin
$_DVWA['recaptcha_public_key'] = getenv('RECAPTCHA_PUBLIC_KEY') ? '' : '';
$_DVWA['recaptcha_private_key'] = getenv('RECAPTCHA_PRIVATE_KEY') ? '' : '';

# Default security level
# Default value for the security level with each session.
# The default is 'impossible'. You may wish to set this to either 'low', 'medium', 'high' or 'impossible'.
$_DVWA['default_security_level'] = getenv('DEFAULT_SECURITY_LEVEL') ? 'impossible' : 'impossible';

# Default locale
```

STEP 6: Fix permissions

sudo chown -R www-data:www-data /var/www/html/DVWA

chmod -R 755 /var/www/html/DVWA

```
(kali@kali)-[/var/www/html]
$ cd /var/www/html/DVWA/config
$ sudo cp config.inc.php.dist config.inc.php
$ sudo nano config.inc.php

(kali@kali)-[/var/www/html/DVWA/config]
$ sudo chown -R www-data:www-data /var/www/html/DVWA
$ sudo chmod -R 755 /var/www/html/DVWA
```

STEP 7: Enable Apache rewrite

sudo a2enmod rewrite

/etc/apache2/apache2.conf

Change: AllowOverride None

to: AllowOverride All

```
(kali@kali)-[/var/www/html/DVWA/config]
$ sudo a2enmod rewrite
$ sudo nano /etc/apache2/apache2.conf
Module rewrite already enabled
```

Restart Apache:

sudo systemctl restart apache2

```
(kali@kali)-[/var/www/html/DVWA/config]
$ sudo systemctl restart apache2
```

STEP 8: Create DVWA database (Browser)

Open browser: <http://localhost/DVWA/setup.php>



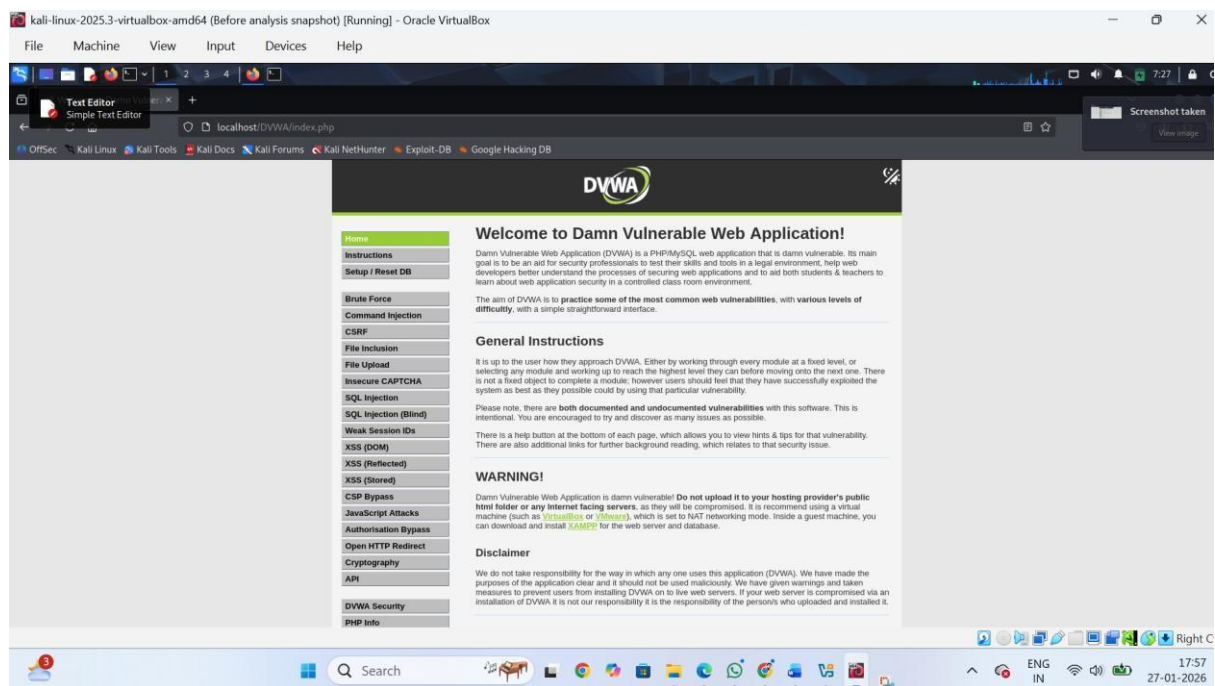
STEP 9: Login to DVWA

<http://localhost/DVWA/login.php>

Credentials:

Username: admin

Password: password



2. OWASP Juice Shop

OWASP Juice Shop is a modern vulnerable web application written in JavaScript (Node.js). It represents real-world application behavior and covers almost all OWASP Top 10 vulnerabilities.

Step 1: Install Docker

```
sudo apt update
sudo apt install docker.io -y
sudo systemctl enable docker
sudo systemctl start docker
```

```
(kali@kali)~$ sudo apt update
Get:1 http://ftp.halifax.rwth-aachen.de/kali kali-rolling InRelease [34.0 kB]
Get:2 http://ftp.halifax.rwth-aachen.de/kali kali-rolling/main amd64 Packages [20.7 MB]
Get:3 http://ftp.halifax.rwth-aachen.de/kali kali-rolling/main amd64 Contents (deb) [52.1 MB]
Fetched 72.8 MB in 21s (3,424 kB/s)
1974 packages can be upgraded. Run 'apt list --upgradable' to see them.
docker.io is already the newest version (27.5.1+dfsg4-1).
The following package was automatically installed and is no longer required:
  libconfig-inifiles-perl
Use 'sudo apt autoremove' to remove it.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 1974
Synchronizing state of docker.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable docker
```

Add user to docker group:

```
sudo usermod -aG docker $USER
docker
```

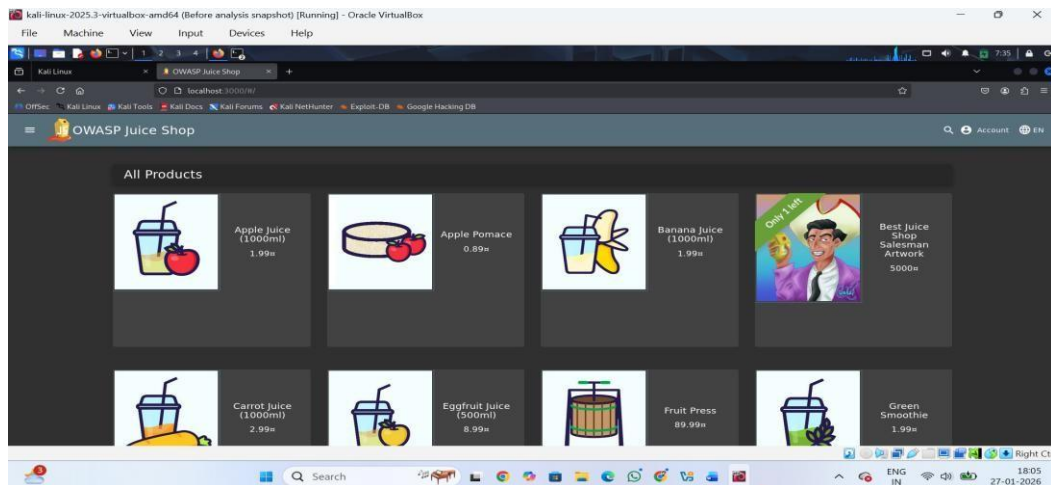
Step 2: Run Juice Shop

```
docker run --rm -p 3000:3000 bkimminich/juice-shop
```

```
(kali@kali)~$ sudo usermod -aG docker $USER
newgrp docker
(kali@kali)~$ docker run --rm -p 3000:3000 bkimminich/juice-shop
info: Detected Node.js version v22.21.1 (OK)
info: Detected OS linux (OK)
info: Detected CPU x64 (OK)
info: Configuration default validated (OK)
info: Entity models 20 of 20 are initialized (OK)
info: Required file server.js is present (OK)
info: Required file styles.css is present (OK)
info: Required file runtime.js is present (OK)
info: Required file vendor.js is present (OK)
info: Required file main.js is present (OK)
info: Required file tutorial.js is present (OK)
info: Required file index.html is present (OK)
info: Port 3000 is available (OK)
info: Chatbot training data botDefaultTrainingData.json validated (OK)
info: Domain https://www.alchemy.com/ is reachable (OK)
info: Server listening on port 3000
```

Step 3: Open browser

<http://localhost:3000>



3. Intercept requests using Burp.

Burp Suite is a web application security testing tool used to:

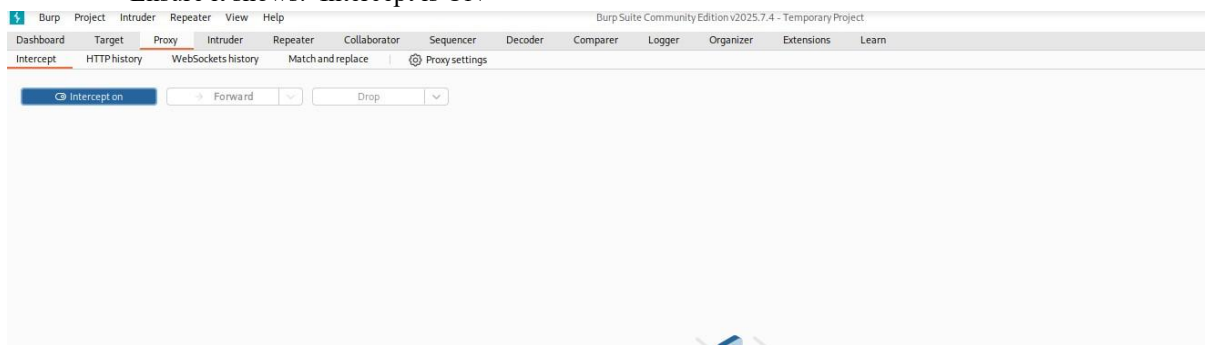
- Intercept HTTP/HTTPS requests
- Modify requests before they reach the server
- Analyze responses
- Find vulnerabilities (SQLi, XSS, Auth bypass, IDOR, etc.)

STEP 1: Open Burp Suite (GUI)

- Click Applications → Web Application Analysis → Burp Suite
- Select Temporary Project
- Click Next → Start Burp

STEP 2: Open Burp Proxy Tab

- Click Proxy
- Click Intercept
- Ensure it shows: Intercept is ON



STEP 3: Configure Firefox Proxy (GUI)

Open Firefox

Click ☰ Menu → Settings

Scroll → Network Settings

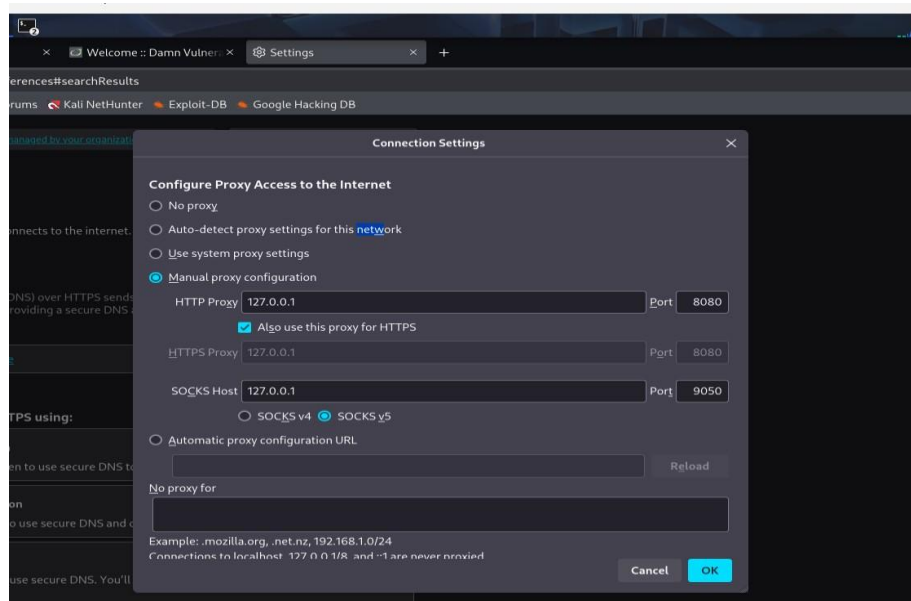
Click Settings

Select Manual proxy configuration Enter:

HTTP Proxy: 127.0.0.1

Port: 8080

Tick Use this proxy for all protocols

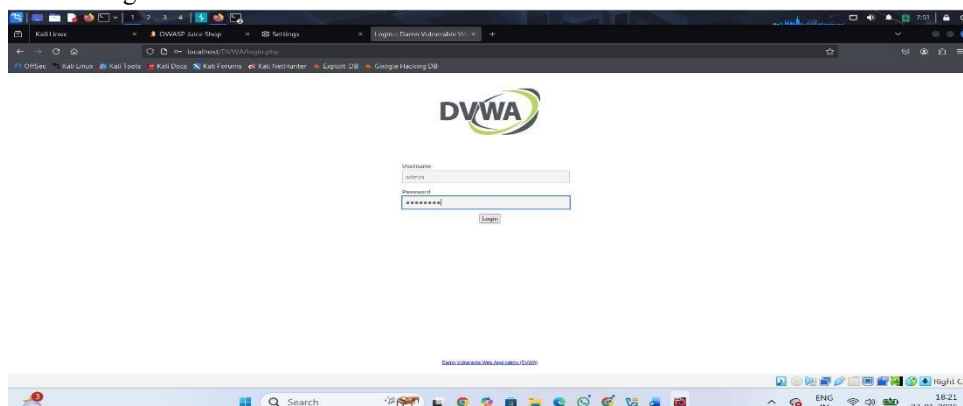


STEP 4: Open DVWA Login Page

In Firefox: <http://localhost/DVWA/login.php>

Enter: Username: admin Password: password

Click Login

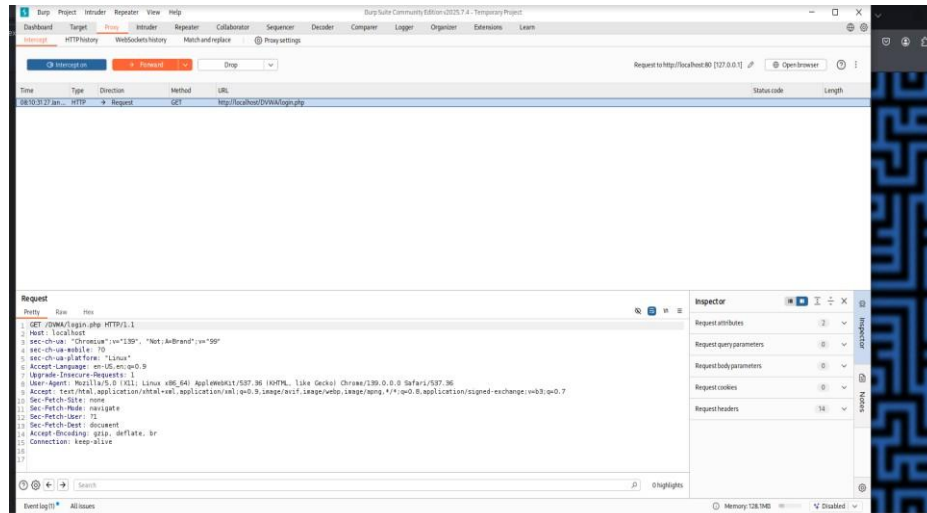


STEP 5: View Intercepted Request (GUI)

Switch to Burp Suite → Proxy → Intercept You
will see a request like:

POST /DVWA/login.php HTTP/1.1

username=admin&password=password&Login=Login



4. Test SQL Injection.

SQL Injection is a web application vulnerability where an attacker manipulates SQL queries by injecting malicious input into user-controlled fields (like login forms, search boxes, URLs).

Why SQL Injection Happens

- User input is directly concatenated into SQL queries
- No input validation or sanitization
- No prepared statements (parameterized queries)
- Poor error handling

STEP Login to DVWA

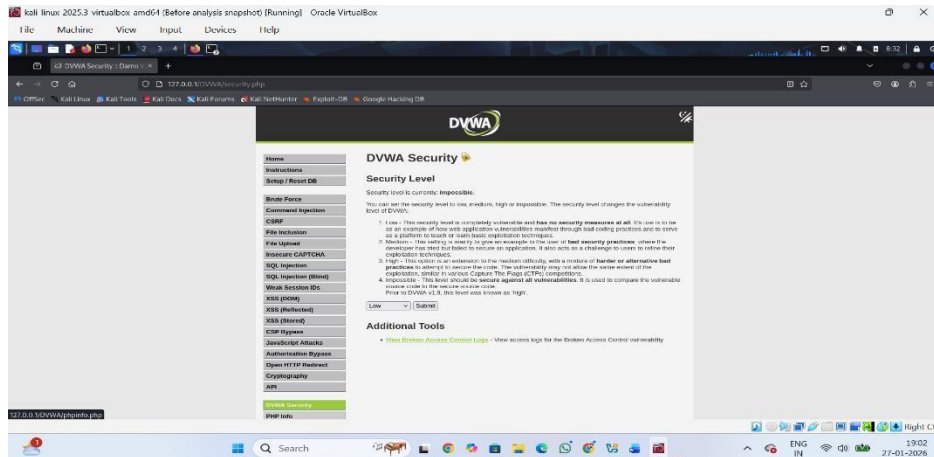
Username: admin

Password: password



STEP Set Security to LOW

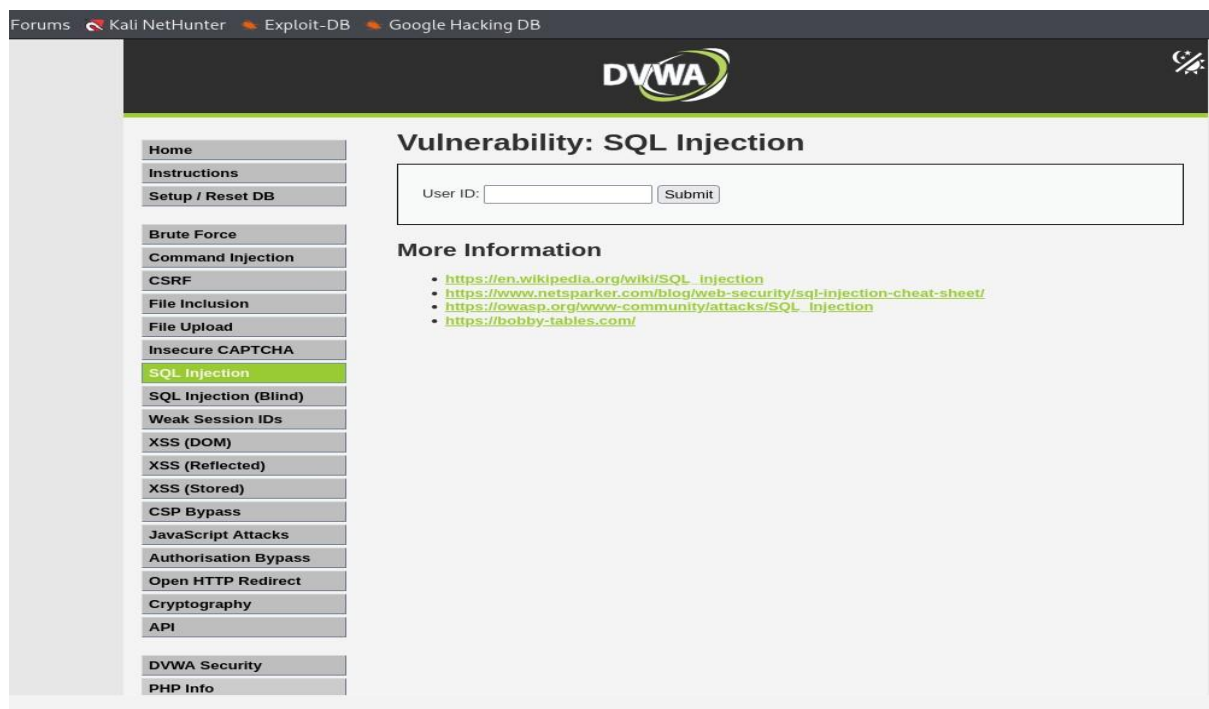
1. Click DVWA Security
2. Set Security Level → LOW
3. Click Submit



STEP Open SQL Injection Page

Go to:

DVWA → Vulnerabilities → SQL Injection



STEP Test Injection (Main Step)

In ID, enter:

1 OR 1=1

Click Submit



STEP Result (SUCCESS)

All user records are displayed

SQL Injection confirmed

5. Test XSS

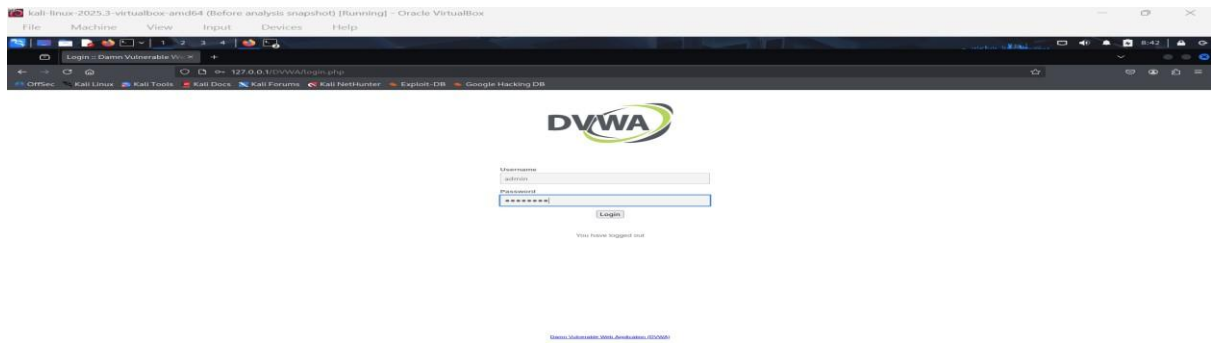
XSS is a web vulnerability where an attacker injects malicious JavaScript into a web page, which then runs in another user's browser. **Why XSS Happens**

- No input validation
- User input is directly shown on web pages
- No output encoding
- No Content Security Policy (CSP)

STEP Login to DVWA

Username: admin

Password: password



STEP Set DVWA Security to LOW

Click DVWA Security

Set Security Level → LOW

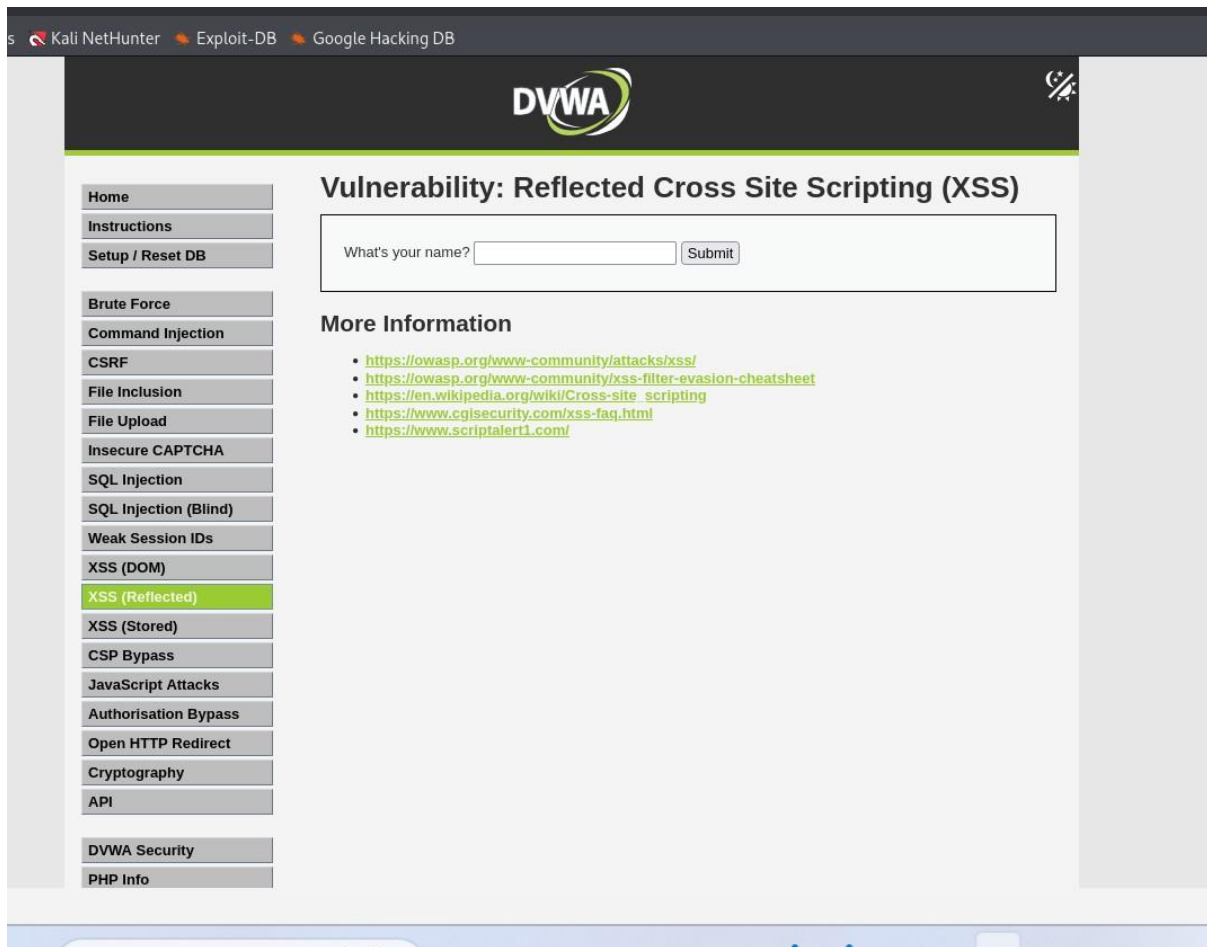
Click Submit



STEP Open XSS (Reflected)

Go to:

DVWA → Vulnerabilities → XSS (Reflected)

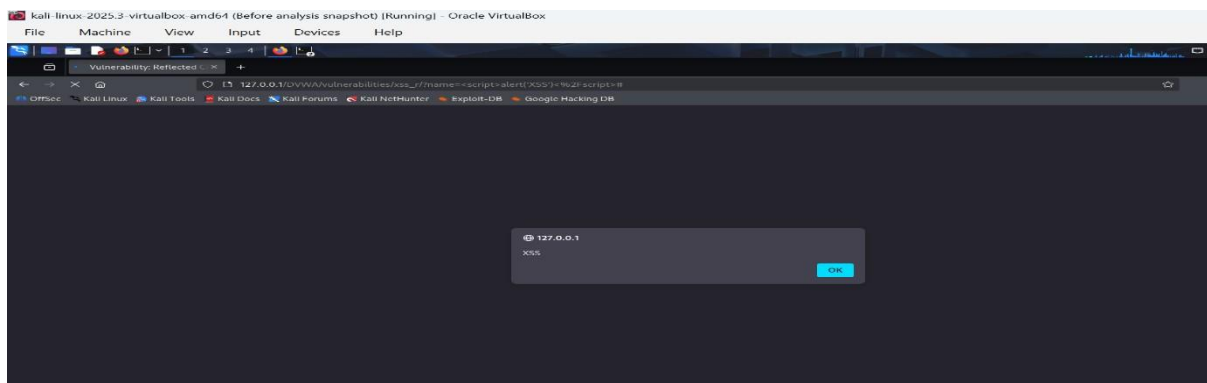


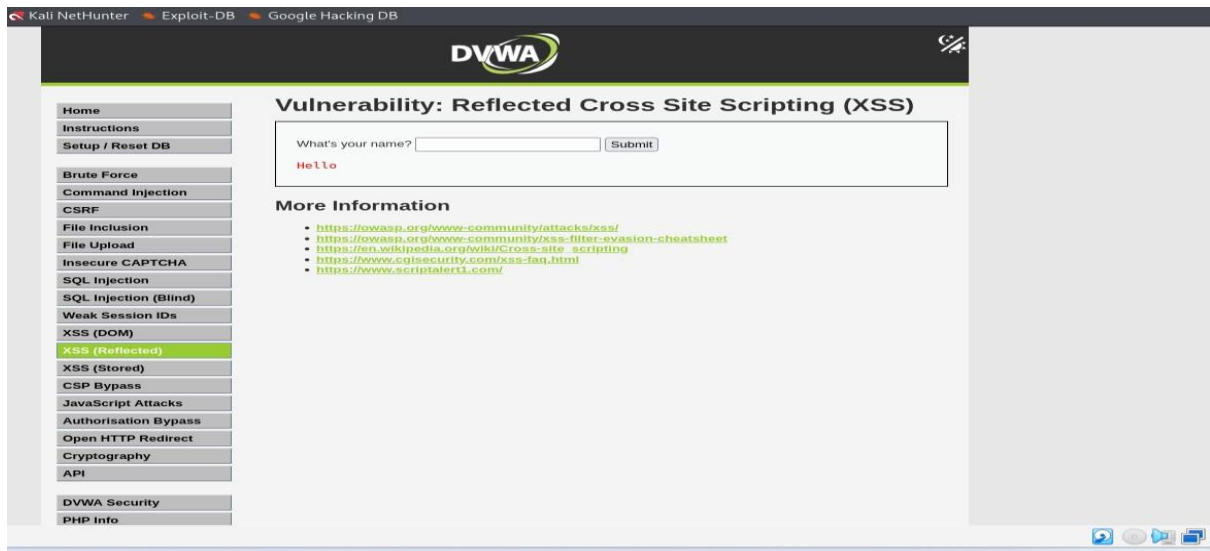
STEP Enter Test Payload

In **input box**, type:

`<script>alert('XSS')</script>`

Click **Submit**





6.Observe Application Responses

During testing, application responses were carefully observed using Burp suite and the browser.

Observations for SQL Injection

- Application returned multiple database records for a single input.
- No input validation or sanitization was applied.
- SQL error messages or abnormal responses were visible.
- HTTP response code remained 200 OK even for malicious input.
- Database queries were modified by user input.

Conclusion:

The application is vulnerable to SQL Injection due to improper handling of user input.

Observations for XSS

- Injected JavaScript executed successfully in the browser.
- Alert box popped up indicating script execution.
- Script was stored and executed for multiple users (Stored XSS).
- No output encoding was implemented.
- User input was directly reflected in the response.

Conclusion:

The application is vulnerable to Cross-Site Scripting (XSS).

7. Document Vulnerabilities

Vulnerability 1: SQL Injection

- Vulnerability Name is as : SQL Injection
- OWASP Category is as : A03 – Injection
- Affected Component: Login / ID input field
- Payload Use is as the following : 1' OR 1=1-- -

Result: Unauthorized access to database records.

Impact: Data leakage will occur

Vulnerability 2: Cross-Site Scripting (XSS)

- Vulnerability Name: Stored / Reflected XSS
- OWASP Category: A07 – Cross-Site Scripting • Affected Component: User input fields
- Payload Used:
- `<script>alert('XSS')</script>`
- Result: JavaScript executed in user's browser.
- Impact: Session hijacking

8. Suggest Mitigations (Theory)

Suggesting mitigations involves recommending security measures to prevent or reduce the risk of identified vulnerabilities. Mitigations are based on secure coding and best security practices.

For **SQL Injection**, mitigations include:

- Using parameterized queries
- Validating and sanitizing user inputs
- Restricting database privileges
- Avoiding detailed error messages

For **Cross-Site Scripting (XSS)**, mitigations include:

- Encoding output before displaying it
- Sanitizing user input

9. Conclusion

Observing responses, documenting vulnerabilities, and suggesting mitigations are essential phases of a secure web application testing process. These steps help in identifying weaknesses, understanding their impact, and strengthening the application against cyber attacks.

